

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
  <title>PF Lift & Swim Tracker</title>
  <!-- Tailwind CSS -->
  <script src="https://cdn.tailwindcss.com"></script>
  <script>
    tailwind.config = {
      theme: {
        extend: {
          colors: {
            pfPurple: '#6d28d9', // Planet Fitness Purple
            pfYellow: '#f59e0b', // Planet Fitness Yellow
          }
        }
      }
    }
  </script>
  <!-- Lucide Icons -->
  <script src="https://unpkg.com/lucide@latest"></script>
  <!-- Firebase SDKs -->
  <script type="module">
    import { initializeApp } from
"https://www.gstatic.com/firebasejs/11.6.1/firebase-app.js";
    import { getAuth, signInWithCustomToken, signInAnonymously,
onAuthStateChanged } from
"https://www.gstatic.com/firebasejs/11.6.1/firebase-auth.js";
    import { getFirestore, collection, addDoc, getDocs } from
"https://www.gstatic.com/firebasejs/11.6.1/firebase-firestore.js";

    window.firebaseImports = { initializeApp, getAuth,
signInWithCustomToken, signInAnonymously, onAuthStateChanged,
getFirestore, collection, addDoc, getDocs };
  </script>
</head>
<body class="bg-slate-950 text-slate-100 font-sans min-h-screen pb-24
antialiased selection:bg-purple-500/30">

  <!-- App Shell Container -->
  <div class="max-w-md mx-auto min-h-screen flex flex-col
bg-slate-900 shadow-2xl relative border-x border-slate-800">

    <!-- Header -->
    <header class="sticky top-0 z-30 bg-slate-900/90
backdrop-blur-md border-b border-slate-800 px-4 py-3 flex

```

```

justify-between items-center">
    <div class="flex items-center gap-2">
        <div class="w-8 h-8 rounded-lg bg-gradient-to-tr
from-purple-600 to-amber-500 flex items-center justify-center
shadow-lg shadow-purple-500/20">
            <i data-lucide="dumbbell" class="w-5 h-5
text-white"></i>
        </div>
        <div>
            <h1 class="text-md font-bold tracking-tight
text-white flex items-center gap-1">PF Gym & Swim</h1>
            <p class="text-[10px] text-slate-400 font-medium
tracking-wide uppercase" id="app-status">Connecting to Cloud...</p>
        </div>
    </div>
    <div class="flex items-center gap-2">
        <span id="sync-indicator" class="w-2.5 h-2.5
rounded-full bg-amber-500 animate-pulse" title="Sync Status"></span>
        <span class="text-xs text-slate-400 font-mono"
id="current-date-badge"></span>
    </div>
</header>

<!-- Main Content View Area -->
<main class="flex-1 p-4 overflow-y-auto">

    <!-- Toast Notifications Area -->
    <div id="toast-box" class="fixed bottom-20 left-1/2
-translate-x-1/2 z-50 pointer-events-none transition-all duration-300
opacity-0 transform translate-y-2">
        <div class="bg-slate-800 text-white border
border-slate-700 px-4 py-2.5 rounded-xl shadow-2xl flex items-center
gap-2 text-xs font-semibold">
            <span id="toast-icon"></span>
            <span id="toast-message">Logged!</span>
        </div>
    </div>

    <!-- TAB 1: TODAY'S WORKOUT VIEW -->
    <section id="view-workout" class="space-y-4">
        <!-- Day Selector Pills -->
        <div class="flex gap-1.5 overflow-x-auto pb-1
no-scrollbar">
            <button onclick="selectDay('Monday')"
class="day-btn shrink-0 text-xs px-3.5 py-2 rounded-xl font-bold
transition-all border border-slate-800 bg-slate-950
text-slate-400">Mon</button>
            <button onclick="selectDay('Tuesday')"
```

```

class="day-btn shrink-0 text-xs px-3.5 py-2 rounded-xl font-bold
transition-all border border-slate-800 bg-slate-950
text-slate-400">Tue</button>
      <button onclick="selectDay('Wednesday') "
class="day-btn shrink-0 text-xs px-3.5 py-2 rounded-xl font-bold
transition-all border border-slate-800 bg-slate-950
text-slate-400">Wed</button>
      <button onclick="selectDay('Thursday') "
class="day-btn shrink-0 text-xs px-3.5 py-2 rounded-xl font-bold
transition-all border border-slate-800 bg-slate-950
text-slate-400">Thu</button>
      <button onclick="selectDay('Friday') "
class="day-btn shrink-0 text-xs px-3.5 py-2 rounded-xl font-bold
transition-all border border-slate-800 bg-slate-950
text-slate-400">Fri</button>
      <button onclick="selectDay('Saturday') "
class="day-btn shrink-0 text-xs px-3.5 py-2 rounded-xl font-bold
transition-all border border-slate-800 bg-slate-950
text-slate-400">Sat</button>
      <button onclick="selectDay('Sunday') "
class="day-btn shrink-0 text-xs px-3.5 py-2 rounded-xl font-bold
transition-all border border-slate-800 bg-slate-950
text-slate-400">Sun</button>
    </div>

    <!-- Current Day Workout Card Header -->
    <div class="bg-gradient-to-br from-slate-800
to-slate-900 rounded-2xl border border-slate-800 p-4 relative
overflow-hidden">
      <div class="absolute -right-6 -bottom-6 opacity-5
pointer-events-none text-slate-400">
        <i data-lucide="dumbbell" class="w-24
h-24"></i>
      </div>
      <div class="flex justify-between items-start">
        <div>
          <span class="text-[9px] font-extrabold
uppercase tracking-widest text-purple-400 px-2 py-0.5 bg-purple-500/10
rounded-md" id="workout-category">Push Day</span>
          <h2 class="text-xl font-black text-white
mt-1" id="workout-day-title">Monday Workout</h2>
        </div>
        <div class="text-right">
          <span class="text-xs text-slate-400
font-medium" id="workout-target-summary">Chest, Shoulders,
Triceps</span>
        </div>
      </div>
    </div>

```

```

</div>

<!-- Exercise List Wrapper -->
<div id="exercise-list-container" class="space-y-4">
  <!-- Loaded dynamically via JavaScript -->
</div>

<!-- Complete Button -->
<div class="pt-2">
  <button onclick="submitWorkoutSession()"
class="w-full bg-gradient-to-r from-purple-600 to-indigo-600
hover:from-purple-500 hover:to-indigo-500 text-white font-bold py-4
rounded-xl shadow-lg shadow-purple-600/15 flex items-center
justify-center gap-2 transition-all duration-150 active:scale-98">
  <i data-lucide="check-circle-2" class="w-5
h-5"></i>
      Finish & Save Session
    </button>
  </div>
</section>

<!-- TAB 2: HISTORY & PROGRESS VIEW -->
<section id="view-history" class="space-y-4 hidden">
  <div class="bg-slate-800/50 rounded-2xl p-4 border
border-slate-800">
    <h2 class="text-lg font-extrabold text-white flex
items-center gap-2">
      <i data-lucide="trending-up" class="w-5 h-5
text-amber-400"></i>
      Strength Trends
    </h2>
    <p class="text-xs text-slate-400 mt-0.5">Select
any machine to view your logged weights over time.</p>

    <!-- Exercise Filter Selector -->
    <select id="exercise-trend-select"
onchange="renderTrendChart()" class="w-full mt-3 bg-slate-950 border
border-slate-800 text-slate-200 rounded-xl px-3 py-2.5 text-xs
font-semibold focus:border-purple-500 focus:outline-none">
      <!-- Dynamic Options loaded via JavaScript -->
    </select>

    <!-- SVG Progression Chart Canvas -->
    <div class="mt-4 h-40 bg-slate-950 rounded-xl
border border-slate-800 p-2 flex flex-col justify-between relative">
      <div id="chart-no-data" class="absolute
inset-0 flex items-center justify-center bg-slate-950/80 rounded-xl
text-xs text-slate-500">

```

```

                No logs found yet. Start saving sessions!
            </div>
            <svg id="trend-svg" class="w-full h-full p-2"
viewBox="0 0 100 100" preserveAspectRatio="none">
                <!-- SVG Path elements generated
dynamically -->
                </svg>
                <div class="flex justify-between text-[8px]
text-slate-500 px-1 font-mono">
                    <span id="chart-start-date">First
Log</span>
                    <span id="chart-end-date">Latest
Log</span>
                </div>
            </div>
        </div>
    </div>

    <div class="space-y-3">
        <h3 class="text-sm font-bold text-slate-400
uppercase tracking-wider flex items-center gap-1.5">
            <i data-lucide="calendar" class="w-4 h-4"></i>
            Past Sessions
        </h3>
        <div id="history-logs-container"
class="space-y-3">
            <!-- Loaded dynamically from Firebase -->
            <div class="p-6 text-center text-xs
text-slate-500 border border-dashed border-slate-800 rounded-xl">
                Connecting database to load your fitness
                history...
            </div>
        </div>
    </div>
</section>

<!-- TAB 3: GUIDE VIEW -->
<section id="view-guide" class="space-y-4 hidden">
    <div class="bg-gradient-to-br from-slate-800
to-slate-900 border border-slate-800 rounded-2xl p-4">
        <h2 class="text-lg font-black text-white flex
items-center gap-2">
            <i data-lucide="help-circle" class="w-5 h-5
text-purple-400"></i>
            PF Machine Reference
        </h2>
        <p class="text-xs text-slate-300 mt-1">Quick tips
to master your weight progression safely on the gym floor.</p>
    </div>

```

```

        <div class="space-y-3">
            <!-- Rule 1 Card -->
            <div class="bg-slate-800/40 p-4 border
border-slate-800 rounded-xl">
                <h3 class="text-xs font-bold text-amber-400
flex items-center gap-1">
                    <span class="w-1.5 h-1.5 rounded-full
bg-amber-400"></span>
                        The "Half-Weight" Dial Hook
                    </h3>
                    <p class="text-xs text-slate-400 mt-1
leading-relaxed">
                        Most machine stacks at Planet Fitness have
standard jumps of 10 lbs or 15 lbs. On the top of the stack, look for
a small slider, selector dial, or a tiny add-on plate. These are
**micro-adjusters of 2.5 lbs or 5 lbs**. Always use these to make
tiny, joint-friendly increments!
                    </p>
                </div>

            <!-- Rule 2 Card -->
            <div class="bg-slate-800/40 p-4 border
border-slate-800 rounded-xl">
                <h3 class="text-xs font-bold text-purple-400
flex items-center gap-1">
                    <span class="w-1.5 h-1.5 rounded-full
bg-purple-400"></span>
                        The Double Progression Rule
                    </h3>
                    <p class="text-xs text-slate-400 mt-1
leading-relaxed">
                        Master your *reps* before touching the
weight. If an exercise is scheduled for **8-12 reps**, do not increase
the weight until you can easily complete **all 12 reps on all 3 sets**
with perfect form. Once you achieve this, add the smallest weight
increment, and drop back down to 8 reps.
                    </p>
                </div>

            <!-- Rule 3 Card -->
            <div class="bg-slate-800/40 p-4 border
border-slate-800 rounded-xl">
                <h3 class="text-xs font-bold text-blue-400
flex items-center gap-1">
                    <span class="w-1.5 h-1.5 rounded-full
bg-blue-400"></span>
                        Recommended Safe Increases

```

```

        </h3>
        <div class="grid grid-cols-2 gap-2 mt-2">
            <div class="bg-slate-950 p-2 rounded-lg
border border-slate-800">
                <span class="text-[10px]
text-slate-500 uppercase font-bold block">Leg Machines</span>
                <span class="text-xs font-black
text-slate-200">Add 5 to 10 lbs</span>
            </div>
            <div class="bg-slate-950 p-2 rounded-lg
border border-slate-800">
                <span class="text-[10px]
text-slate-500 uppercase font-bold block">Chest & Back</span>
                <span class="text-xs font-black
text-slate-200">Add 5 lbs</span>
            </div>
            <div class="bg-slate-950 p-2 rounded-lg
border border-slate-800">
                <span class="text-[10px]
text-slate-500 uppercase font-bold block">Shoulders & Arms</span>
                <span class="text-xs font-black
text-slate-200">Add 2.5 lbs (Cables)</span>
            </div>
            <div class="bg-slate-950 p-2 rounded-lg
border border-slate-800">
                <span class="text-[10px]
text-slate-500 uppercase font-bold block">Dumbbell Weights</span>
                <span class="text-xs font-black
text-slate-200">Aim for 15 reps first</span>
            </div>
        </div>
    </div>
</div>
</section>

</main>

<!-- Bottom Tab Bar Navigation -->
<nav class="fixed bottom-0 left-0 right-0 max-w-md mx-auto
bg-slate-950 border-t border-slate-850 px-6 py-2 flex justify-between
items-center z-40">
    <button onclick="switchTab('workout')" id="nav-workout"
class="flex flex-col items-center gap-1 text-purple-500
transition-colors py-1 px-3">
        <i data-lucide="dumbbell" class="w-5 h-5"></i>
        <span class="text-[10px] font-bold">Lifting Day</span>
    </button>
    <button onclick="switchTab('history')" id="nav-history"

```

```

class="flex flex-col items-center gap-1 text-slate-500
transition-colors py-1 px-3">
    <i data-lucide="trending-up" class="w-5 h-5"></i>
    <span class="text-[10px] font-bold">Trends &
Logs</span>
    </button>
    <button onclick="switchTab('guide')" id="nav-guide"
class="flex flex-col items-center gap-1 text-slate-500
transition-colors py-1 px-3">
    <i data-lucide="help-circle" class="w-5 h-5"></i>
    <span class="text-[10px] font-bold">PF Guide</span>
    </button>
</nav>

</div>

<!-- Application Script & Data Management -->
<script type="module">
    // Workout Data Schema structured for Planet Fitness machine
alignment (no bodyweight)
    const workoutProgram = {
        Monday: {
            title: "Monday: Push Day",
            category: "Lifting",
            target: "Chest, Shoulders, Triceps",
            exercises: [
                { name: "Dumbbell Chest Press", target: "3 Sets x
8 - 10 Reps", sets: 3 },
                { name: "Seated DB Shoulder Press", target: "3
Sets x 10 - 12 Reps", sets: 3 },
                { name: "Machine Chest Fly (Pec Dec)", target: "3
Sets x 12 Reps", sets: 3 },
                { name: "Dumbbell Lateral Raises", target: "3 Sets
x 12 - 15 Reps", sets: 3 },
                { name: "Tricep Rope Pushdowns", target: "3 Sets x
12 Reps", sets: 3 }
            ]
        },
        Tuesday: {
            title: "Tuesday: Pull Day",
            category: "Lifting",
            target: "Back, Rear Delts, Biceps",
            exercises: [
                { name: "Lat Pulldowns (Wide Grip)", target: "4
Sets x 8 - 10 Reps", sets: 4 },
                { name: "Seated Cable Rows", target: "3 Sets x 10
Reps", sets: 3 },
                { name: "Lat Pulldowns (Underhand)", target: "3

```



```

Sets x 12 Reps", sets: 3 },
      { name: "Face Pulls (Rear Delts)", target: "3 Sets
x 15 Reps", sets: 3 },
      { name: "Dumbbell Bicep Curls", target: "3 Sets x
10 - 12 Reps", sets: 3 }
    ]
  },
  Wednesday: {
    title: "Wednesday: Swim Day",
    category: "Swim",
    target: "Zero-Impact Active Recovery",
    exercises: [
      { name: "Leisurely Rec Swim", target: "20 to 30
Mins (Easy Laps)", sets: 1, isSwim: true }
    ]
  },
  Thursday: {
    title: "Thursday: Leg Day",
    category: "Lifting",
    target: "Quads, Hamstrings, Calves",
    exercises: [
      { name: "Leg Press Machine", target: "3 Sets x 8 -
10 Reps", sets: 3 },
      { name: "Romanian Deadlifts (Dumbbell)", target:
"3 Sets x 10 Reps", sets: 3 },
      { name: "Seated/Lying Leg Curl Machine", target:
"3 Sets x 10 - 12 Reps", sets: 3 },
      { name: "Machine Hip Abductor (Outer)", target: "3
Sets x 12 - 15 Reps", sets: 3 },
      { name: "Calf Press (On Leg Press)", target: "3
Sets x 15 Reps", sets: 3 }
    ]
  },
  Friday: {
    title: "Friday: Core & Arms",
    category: "Lifting",
    target: "Abs, Obliques, Arms",
    exercises: [
      { name: "Hammer Curls (Dumbbell)", target: "3 Sets
x 10 - 12 Reps", sets: 3 },
      { name: "Overhead Tricep Extension (DB)", target:
"3 Sets x 12 Reps", sets: 3 },
      { name: "Seated Ab Crunch Machine", target: "3
Sets x 12 - 15 Reps", sets: 3 },
      { name: "Seated Torso Rotation Machine", target:
"3 Sets x 12 - 15 Reps", sets: 3 }
    ]
  },

```

```

        Saturday: {
            title: "Saturday: Swim Day",
            category: "Swim",
            target: "Leg Soreness Flush Recovery",
            exercises: [
                { name: "Leisurely Rec Swim", target: "20 to 30
Mins (Easy Laps)", sets: 1, isSwim: true }
            ]
        },
        Sunday: {
            title: "Sunday: Swim Day",
            category: "Swim",
            target: "Joint & Spine Decompression",
            exercises: [
                { name: "Leisurely Rec Swim", target: "20 to 30
Mins (Easy Laps)", sets: 1, isSwim: true }
            ]
        }
    };

    let currentDay = "Monday";
    let activeTab = "workout";
    let db, auth, appId, userId;
    let isFirebaseReady = false;
    let localWorkoutHistory = [];

    // UI Initializer
    window.addEventListener('DOMContentLoaded', async () => {
        lucide.createIcons();
        setCurrentDateBadge();
        detectAndSetCurrentDay();
        renderSelectedWorkout();
        await initFirebase();
    });

    // Initialize Secure Firebase Connection
    async function initFirebase() {
        const statusText = document.getElementById('app-status');
        const syncDot = document.getElementById('sync-indicator');

        try {
            // Config fallback safety
            if (typeof __firebase_config === 'undefined') {
                console.warn("Using offline demo database. Provide
Firebase variables to synchronize.");
                statusText.innerText = "Offline Mode (Local)";
                syncDot.className = "w-2.5 h-2.5 rounded-full
bg-blue-500";
            }
        }
    }

```

```

        loadOfflineHistory();
        return;
    }

    const { initializeApp, getAuth, signInWithCustomToken,
signInAnonymously, onAuthStateChanged, getFirestore } =
window.firebaseImports;

    const firebaseConfig = JSON.parse(__firebase_config);
    const app = initializeApp(firebaseConfig);
    auth = getAuth(app);
    db = getFirestore(app);
    appId = typeof __app_id !== 'undefined' ? __app_id :
'pf-workout-tracker-default';

    // Rule 3: Auth first
    if (typeof __initial_auth_token !== 'undefined' &&
__initial_auth_token) {
        await signInWithCustomToken(auth,
__initial_auth_token);
    } else {
        await signInAnonymously(auth);
    }

    onAuthStateChanged(auth, async (user) => {
        if (user) {
            userId = user.uid;
            isFirebaseReady = true;
            statusText.innerText = `Connected (Cloud)`;
            syncDot.className = "w-2.5 h-2.5 rounded-full
bg-emerald-500";

            await fetchCloudHistory();
        }
    });

    } catch (err) {
        console.error("Firebase startup crash:", err);
        statusText.innerText = "Error (Local Backup Enabled)";
        syncDot.className = "w-2.5 h-2.5 rounded-full
bg-red-500 animate-none";
        loadOfflineHistory();
    }
}

// Set Date Indicator Badges
function setCurrentDateBadge() {
    const days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday',
'Thursday', 'Friday', 'Saturday'];

```

```

        const months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
        const d = new Date();
        const badge =
document.getElementById('current-date-badge');
        badge.innerText = `${days[d.getDay()]},
${months[d.getMonth()]} ${d.getDate()}`;
    }

    // Auto-detect Day of the week on load
    function detectAndSetCurrentDay() {
        const days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday',
'Thursday', 'Friday', 'Saturday'];
        const todayName = days[new Date().getDay()];
        currentDay = todayName;
        updateDaySelectorHighlight();
    }

    // Toggle Tabs (Vanilla JS hide/show instead of complex
routing)
    window.switchTab = function(tabName) {
        activeTab = tabName;

document.getElementById('view-workout').classList.add('hidden');

document.getElementById('view-history').classList.add('hidden');

document.getElementById('view-guide').classList.add('hidden');

document.getElementById(`view-${tabName}`).classList.remove('hidden');

        // Toggle Icon highlights
        ['workout', 'history', 'guide'].forEach(t => {
            const navBtn = document.getElementById(`nav-${t}`);
            if (t === tabName) {
                navBtn.className = "flex flex-col items-center
gap-1 text-purple-400 transition-colors py-1 px-3";
            } else {
                navBtn.className = "flex flex-col items-center
gap-1 text-slate-500 transition-colors py-1 px-3";
            }
        });

        if (tabName === 'history') {
            populateTrendDropdown();
            renderTrendChart();
        }
    }

```

```

    }

    // Select specific weekday
    window.selectDay = function(dayName) {
        currentDay = dayName;
        updateDaySelectorHighlight();
        renderSelectedWorkout();
    }

    // Update Pill highlights
    function updateDaySelectorHighlight() {
        const btns = document.querySelectorAll('.day-btn');
        const dayAbbrs = { Monday: "Mon", Tuesday: "Tue",
Wednesday: "Wed", Thursday: "Thu", Friday: "Fri", Saturday: "Sat",
Sunday: "Sun" };

        btns.forEach(btn => {
            if (btn.innerText === dayAbbrs[currentDay]) {
                btn.className = "day-btn shrink-0 text-xs px-3.5
py-2 rounded-xl font-black transition-all border border-purple-500/30
bg-purple-600/20 text-purple-300";
            } else {
                btn.className = "day-btn shrink-0 text-xs px-3.5
py-2 rounded-xl font-bold transition-all border border-slate-800
bg-slate-950 text-slate-400";
            }
        });
    }

    // Render Current day layout in detail
    function renderSelectedWorkout() {
        const data = workoutProgram[currentDay];
        const listContainer =
document.getElementById('exercise-list-container');

        document.getElementById('workout-day-title').innerText =
data.title;
        document.getElementById('workout-category').innerText =
data.category + " Day";

        document.getElementById('workout-target-summary').innerText =
data.target;

        if (data.category === 'Swim') {
            document.getElementById('workout-category').className
= "text-[9px] font-extrabold uppercase tracking-widest text-sky-400
px-2 py-0.5 bg-sky-500/10 rounded-md";
        } else {

```

```

        document.getElementById('workout-category').className
= "text-[9px] font-extrabold uppercase tracking-widest text-purple-400
px-2 py-0.5 bg-purple-500/10 rounded-md";
    }

    listContainer.innerHTML = '';

    data.exercises.forEach((ex, exIdx) => {
        let cardHTML = '';
        if (ex.isSwim) {
            cardHTML = `
                <div class="bg-slate-950 border
border-slate-800/80 rounded-2xl p-4 flex flex-col justify-between"
id="ex-${exIdx}">
                    <div class="flex justify-between
items-start">
                        <div>
                            <h4 class="font-black
text-slate-200 text-sm">${ex.name}</h4>
                            <span class="text-[10px]
text-sky-400 font-semibold uppercase tracking-wide mt-1
block">${ex.target}</span>
                        </div>
                        <span class="text-[9px] font-bold
text-slate-500 border border-slate-800 rounded px-1.5 py-0.5
uppercase">Pool</span>
                    </div>
                    <div class="mt-4 pt-3 border-t
border-slate-900 flex justify-between items-center">
                        <input type="number"
id="swim-input-mins" placeholder="Mins swam" class="w-28 bg-slate-900
border border-slate-800 text-xs py-2 px-3 rounded-lg text-slate-200
focus:border-sky-500 focus:outline-none">
                        <button onclick="markSwimReady(this)"
class="bg-sky-600/20 text-sky-400 border border-sky-500/30 font-bold
text-xs py-2 px-4 rounded-xl transition-all hover:bg-sky-600/30">Set
Done</button>
                    </div>
                </div>
            `;
        } else {
            let setButtonsHTML = '';
            for (let s = 1; s <= ex.sets; s++) {
                setButtonsHTML += `
                    <button onclick="toggleCardSet(this,
${exIdx}, ${s})" class="set-checker-btn w-10 h-10 rounded-full border
border-slate-800 text-xs font-black flex items-center justify-center
transition-all bg-slate-900 text-slate-400

```

```

hover:border-purple-500/40">
    ${s}
  </button>
  `;
}

cardHTML = `
  <div class="bg-slate-950 border
border-slate-850 rounded-2xl p-4 space-y-3" id="ex-${exIdx}">
    <div class="flex justify-between
items-start">
      <div>
        <h4 class="font-extrabold
text-slate-100 text-sm leading-tight">${ex.name}</h4>
        <span class="text-[10px]
text-slate-400 font-medium tracking-wide block
mt-1">${ex.target}</span>
      </div>
      <span class="text-[9px] font-extrabold
text-slate-500 border border-slate-850 bg-slate-900/60 rounded px-1.5
py-0.5 uppercase">Machine</span>
    </div>

    <div class="flex gap-2 items-center
bg-slate-900/40 p-2 rounded-xl border border-slate-850/50">
      <div class="flex-1">
        <label class="text-[9px] uppercase
font-bold text-slate-500 block">Weight (lbs)</label>
        <input type="number"
id="ex-weight-${exIdx}" placeholder="Weight" class="w-full
bg-slate-950 border border-slate-800 text-xs py-1.5 px-2 rounded-md
text-slate-200 focus:border-purple-500 focus:outline-none font-bold">
      </div>
      <div class="flex-1">
        <label class="text-[9px] uppercase
font-bold text-slate-500 block">Reps Logged</label>
        <input type="number"
id="ex-reps-${exIdx}" placeholder="Reps Done" class="w-full
bg-slate-950 border border-slate-800 text-xs py-1.5 px-2 rounded-md
text-slate-200 focus:border-purple-500 focus:outline-none font-bold">
      </div>
    </div>

    <div class="flex items-center gap-2
pt-1.5">
      <span class="text-[9px] font-black
uppercase text-slate-500 tracking-wider">Tap Sets Completed:</span>
      <div class="flex gap-1.5 ml-1">

```

```

        ${setButtonsHTML}
      </div>
    </div>
  </div>
  `;
}

listContainer.insertAdjacentHTML('beforeend',
cardHTML);
});
lucide.createIcons();
}

// Handle quick tap set highlights
window.toggleCardSet = function(btn, exIdx, setNum) {
  btn.classList.toggle('bg-slate-900');
  btn.classList.toggle('text-slate-400');
  btn.classList.toggle('border-slate-800');

  btn.classList.toggle('bg-emerald-600/20');
  btn.classList.toggle('text-emerald-400');
  btn.classList.toggle('border-emerald-500/40');

  btn.classList.add('scale-95');
  setTimeout(() => btn.classList.remove('scale-95'), 100);
}

// Quick Swim toggles
window.markSwimReady = function(btn) {
  const input = document.getElementById('swim-input-mins');
  if (!input.value) {
    showToast("Enter minutes swam first", "error");
    return;
  }
  btn.innerText = "Done ✓";
  btn.className = "bg-emerald-600/20 text-emerald-400 border
border-emerald-500/30 font-bold text-xs py-2 px-4 rounded-xl";
  showToast("Rec swim prepared for log!", "success");
}

// Log Submit Handler
window.submitWorkoutSession = async function() {
  const currentWorkout = workoutProgram[currentDay];
  const submission = {
    date: new Date().toISOString().split('T')[0],
    timestamp: Date.now(),
    day: currentDay,
    workoutType: currentWorkout.title,
  };

```



```

        category: currentWorkout.category,
        exercises: []
    };

    let validationPassed = false;

    // Loop and compile set items
    currentWorkout.exercises.forEach((ex, idx) => {
        if (ex.isSwim) {
            const mins =
document.getElementById('swim-input-mins').value;
            if (mins) {
                submission.exercises.push({
                    name: ex.name,
                    weight: 0,
                    reps: parseInt(mins),
                    completedSets: 1
                });
                validationPassed = true;
            }
        } else {
            const weightVal =
document.getElementById(`ex-weight-${idx}`).value;
            const repsVal =
document.getElementById(`ex-reps-${idx}`).value;

            // Count completed circles
            const activeSetElements =
document.querySelectorAll(`#ex-${idx} .set-checker-btn`);
            let completedSetCount = 0;
            activeSetElements.forEach(btn => {
                if
(btn.classList.contains('bg-emerald-600/20')) {
                    completedSetCount++;
                }
            });

            if (weightVal && repsVal && completedSetCount > 0)
{
                submission.exercises.push({
                    name: ex.name,
                    weight: parseFloat(weightVal),
                    reps: parseInt(repsVal),
                    completedSets: completedSetCount
                });
                validationPassed = true;
            }
        }
    });
}

```

```

    });

    if (!validationPassed) {
        showToast("Log at least 1 set to submit!", "error");
        return;
    }

    // Save to Firebase (Rule 1: private path)
    if (isFirebaseReady) {
        const { collection, addDoc } = window.firebaseImports;
        try {
            const path = collection(db, 'artifacts', appId,
'users', userId, 'workout_logs');
            await addDoc(path, submission);
            showToast("Workout Saved to Cloud!", "success");
            await fetchCloudHistory();
        } catch (err) {
            console.error("Save failed:", err);
            showToast("Database Save Error. Retrying local.",
"error");

            saveBackupLocal(submission);
        }
    } else {
        saveBackupLocal(submission);
    }

    clearWorkoutFields();
}

// Clean values after submission
function clearWorkoutFields() {
    const currentWorkout = workoutProgram[currentDay];
    currentWorkout.exercises.forEach((ex, idx) => {
        if (ex.isSwim) {
            const input =
document.getElementById('swim-input-mins');
            if (input) input.value = '';
        } else {
            const wInput =
document.getElementById(`ex-weight-${idx}`);
            const rInput =
document.getElementById(`ex-reps-${idx}`);
            if (wInput) wInput.value = '';
            if (rInput) rInput.value = '';

            // Reset circles
            const btns = document.querySelectorAll(`#ex-${idx}
.set-checker-btn`);

```

```

        btns.forEach(btn => {
            btn.className = "set-checker-btn w-10 h-10
rounded-full border border-slate-800 text-xs font-black flex
items-center justify-center transition-all bg-slate-900 text-slate-400
hover:border-purple-500/40";
        });
    }
});
renderSelectedWorkout();
}

// Local Storage fallback protocols
function saveBackupLocal(log) {
    const localHist =
JSON.parse(localStorage.getItem('pf_backup_logs') || '[]');
    localHist.push(log);
    localStorage.setItem('pf_backup_logs',
JSON.stringify(localHist));
    localWorkoutHistory = localHist;
    showToast("Saved Locally!", "success");
    renderHistoryLogs();
}

function loadOfflineHistory() {
    localWorkoutHistory =
JSON.parse(localStorage.getItem('pf_backup_logs') || '[]');
    renderHistoryLogs();
}

// Load logs safely (Rule 2: Fetch simple list, sort in JS)
async function fetchCloudHistory() {
    const { collection, getDocs } = window.firebaseImports;
    try {
        const path = collection(db, 'artifacts', appId,
'users', userId, 'workout_logs');
        const snapshot = await getDocs(path);
        const list = [];
        snapshot.forEach(doc => {
            list.push(doc.data());
        });

        // Rule 2: Sort descending by date in memory
        list.sort((a, b) => b.timestamp - a.timestamp);
        localWorkoutHistory = list;
        renderHistoryLogs();
    } catch (err) {
        console.error("Fetch failure:", err);
        loadOfflineHistory();
    }
}

```

```

    }
  }

  // Render History UI list
  function renderHistoryLogs() {
    const container =
document.getElementById('history-logs-container');
    if (localWorkoutHistory.length === 0) {
      container.innerHTML = `
        <div class="p-8 text-center text-xs text-slate-500
border border-dashed border-slate-800 rounded-xl">
          No logs saved yet. Finish a session to see it
logged here!
        </div>
      `;
      return;
    }

    container.innerHTML = '';
    localWorkoutHistory.forEach((log) => {
      const formattedDate = new
Date(log.timestamp).toLocaleDateString(undefined, { month: 'short',
day: 'numeric', year: 'numeric' });

      let exercisesListHTML = '';
      log.exercises.forEach(ex => {
        if (log.category === 'Swim') {
          exercisesListHTML += `
            <div class="flex justify-between text-xs
text-slate-300">
              <span>${ex.name}</span>
              <span class="font-bold
text-sky-400">${ex.reps} Mins Swam</span>
            </div>
          `;
        } else {
          exercisesListHTML += `
            <div class="flex justify-between text-xs
text-slate-300">
              <span>${ex.name}</span>
              <span class="font-mono
text-slate-400">
                <strong
class="text-white">${ex.weight} lbs</strong> • ${ex.reps} Reps
                (${ex.completedSets} sets)
              </span>
            </div>
          `;
        }
      });
    });
  }

```

```

    }
  });

  const logColor = log.category === 'Swim' ?
'border-sky-500/20 bg-sky-950/10' : 'border-slate-800
bg-slate-950/40';
  const tagColor = log.category === 'Swim' ?
'text-sky-400 bg-sky-500/10' : 'text-purple-400 bg-purple-500/10';

  const logHTML = `
    <div class="p-4 rounded-xl border ${logColor}
space-y-2">
      <div class="flex justify-between items-center
pb-1 border-b border-slate-800">
        <div>
          <h4 class="text-xs font-black
text-slate-100">${log.day} Session</h4>
          <span class="text-[10px]
text-slate-400 font-mono">${formattedDate}</span>
        </div>
        <span class="text-[9px] font-extrabold
px-2 py-0.5 rounded uppercase tracking-wide ${tagColor}">
          ${log.category}
        </span>
      </div>
      <div class="space-y-2 pt-1">
        ${exercisesListHTML}
      </div>
    </div>
  `;
  container.insertAdjacentHTML('beforeend', logHTML);
});
}

// Dynamic chart and dropdown populate
function populateTrendDropdown() {
  const select =
document.getElementById('exercise-trend-select');
  const uniqueExercises = new Set();

  Object.keys(workoutProgram).forEach(day => {
    workoutProgram[day].exercises.forEach(ex => {
      if (!ex.isSwim) uniqueExercises.add(ex.name);
    });
  });

  select.innerHTML = '';
  uniqueExercises.forEach(exName => {

```

```

        const opt = document.createElement('option');
        opt.value = exName;
        opt.innerText = exName;
        select.appendChild(opt);
    });
}

// Plot Progression SVG Path inside view
window.renderTrendChart = function() {
    const exName =
document.getElementById('exercise-trend-select').value;
    const trendData = [];

    // Gather all entries for selected workout
    localWorkoutHistory.forEach(log => {
        log.exercises.forEach(ex => {
            if (ex.name === exName && ex.weight > 0) {
                trendData.push({ timestamp: log.timestamp,
weight: ex.weight });
            }
        });
    });

    const noDataBanner =
document.getElementById('chart-no-data');
    const svg = document.getElementById('trend-svg');

    if (trendData.length < 2) {
        noDataBanner.classList.remove('hidden');
        svg.innerHTML = '';
        return;
    }

    noDataBanner.classList.add('hidden');

    // Sort Chronologically
    trendData.sort((a,b) => a.timestamp - b.timestamp);

    const weights = trendData.map(d => d.weight);
    const maxWeight = Math.max(...weights);
    const minWeight = Math.min(...weights);
    const range = maxWeight - minWeight === 0 ? 10 :
(maxWeight - minWeight);

    // Generate Path Points
    const points = trendData.map((d, i) => {
        const x = (i / (trendData.length - 1)) * 100;
        const y = 90 - ((d.weight - minWeight) / range) * 80;
    });

```

```

// Margin top/bottom inside SVG space
    return `${x},${y}`;
  });

  // Set labels
  document.getElementById('chart-start-date').innerText =
new Date(trendData[0].timestamp).toLocaleDateString(undefined, {
month: 'short', day: 'numeric' });
  document.getElementById('chart-end-date').innerText = new
Date(trendData[trendData.length-1].timestamp).toLocaleDateString(undef
ined, { month: 'short', day: 'numeric' });

  svg.innerHTML = `
    <!-- Grid Lines -->
    <line x1="0" y1="10" x2="100" y2="10" stroke="#1e293b"
stroke-width="0.5" stroke-dasharray="1,1" />
    <line x1="0" y1="50" x2="100" y2="50" stroke="#1e293b"
stroke-width="0.5" stroke-dasharray="1,1" />
    <line x1="0" y1="90" x2="100" y2="90" stroke="#1e293b"
stroke-width="0.5" stroke-dasharray="1,1" />

    <!-- The Trend Line -->
    <polyline fill="none" stroke="#6d28d9"
stroke-width="2.5" stroke-linecap="round" stroke-linejoin="round"
points="${points.join(' ')}" />

    <!-- Glow effect under trend line -->
    <path d="M 0,100 L ${points.map(p => p).join(' L ')} L
100,100 Z" fill="url(#grad)" opacity="0.15" />

    <defs>
      <linearGradient id="grad" x1="0%" y1="0%" x2="0%"
y2="100%">
        <stop offset="0%"
style="stop-color:#8b5cf6;stop-opacity:1" />
        <stop offset="100%"
style="stop-color:#6d28d9;stop-opacity:0" />
      </linearGradient>
    </defs>

    <!-- Data Dot Indicators -->
    ${points.map((p, i) => {
      const coords = p.split(',');
      return `
        <circle cx="${coords[0]}" cy="${coords[1]}"
r="2" fill="#fbbf24" stroke="#020617" stroke-width="0.5" />
        <text x="${coords[0]}" y="${coords[1] - 4}"
fill="#e2e8f0" font-size="5.5" font-family="monospace"

```

```

font-weight="black" text-anchor="middle">
    ${trendData[i].weight}
</text>
    `;
    }).join('')}
    `;
}

// Custom UI Toast Notification (Safer than browser alert)
function showToast(msg, type = "success") {
    const box = document.getElementById('toast-box');
    const messageSpan =
document.getElementById('toast-message');
    const iconSpan = document.getElementById('toast-icon');

    messageSpan.innerText = msg;

    if (type === "success") {
        iconSpan.innerHTML = `

```